Haskell Notes

目次

1 凡例 2
2 Haskell について 2
3 関数型という考え方 3
4 変数・関数・型 3
4.1 変数 3
4.2 変数の型 4
4.3 関数 4
4.4 有名な関数 5
4.5 関数の型 5
4.6 関数合成 5
4.7 IO サバイバルキット 1 6
5 条件分岐と再帰呼び出し 6
5.1 ラムダ 6
5.2 ローカル変数 7
5.3 クロージャ 8
5.4 型 8
6 Test Part 8

1 凡例

種類	字体・表記法	例
変数・関数	イタリック（1 文字）	x, f
有名な変数・関数	ローマン・大文字	$\mathrm{first}, \mathrm{id}$
リスト変数	変数名に s をつける	x_{s}
Maybe 変数	変数名に ? をつける	$x_{?}$
一般のコンテナ変数	変数名に $*$ をつける	x_{*}
定数値コンストラクタ 1	数学記号	$\emptyset$
定数値コンストラクタ 2	ローマン・大文字	True
値コンストラクタ	ローマン・大文字	Just_x
有名な値コンストラクタ	特別なカッコで包む	$[x], \langle x \rangle$
アクション	ギリシア文字（1 文字）	α
有名なアクション	サンセリフ	main
型	ボールドイタリック（1 文字）	$\boldsymbol{a}$
有名な型	ボールド・大文字	$\mathbf{Int}$
型クラス	フラクチュール	$\mathfrak{Num}$
キーワード	固定幅	`let`
集合（数学）	ブラックボード	$\mathbb{Z}$
関手（数学）	筆記体	$\mathscr{F}$

2 Haskell について

本書はプログラミング言語 Haskell の入門書である．それと同時に，本書はプログラミング言語を用いた代数構造の入門書でもある．プログラミングと代数構造の間には密接な関係があるが，とくに関数型プログラミングを実践する時にはその関係を意識する必要が出てくる．本書はその両者を同時に解説することを試みる．

これからのプログラマにとって Haskell を無視することはできない．Haskell の「欠点をあげつらうことも，攻撃することもできるが，無視することだけはできない」のだ．それは Haskell がプログラミングの本質に深く関わっているからである．

Haskell というプログラミング言語を知ろうとすると，従来のプログラミング言語の知識が邪魔をする．モダンで，人気があって，Haskell から影響を受けた言語，たとえば Ruby や Swift の知識さえ，Haskell を学ぶ障害になり得る．ではどのようにして Haskell の深みに到達すればいいのだろうか．

その答えは，一見遠回りに見えるが，一度抽象数学の高みに登ることである．

と言っても，あわてる必要はない．

近代的なプログラミング言語を知っていれば，すでにある程度抽象数学に足を踏み入れているからである．そこで，本書では近代的なプログラマを対象に，プログラミング言語を登山口に抽象数学の山を登り，その高みから Haskell という森を見下ろすことにする．

ところで，プログラムのソースコードは現代でも ASCII 文字セットの範囲で書くことが標準的である．Unicode を利用したり，まして文字にカラーを指定したり，書体や装飾を指定することは一般的ではない．たとえば変数 `a` のことを a と書いたり $\boldsymbol{a}$ と書いたり $\tilde{a}$ と書いたりして区別することはない．

Haskell プログラマもまた，多くの異なる概念を同じ貧弱な文字セットで表現しなければならない．これは，はじめて Haskell コードを読むときに大きな問題になりえる．たとえば Haskell では `[a]` という表記をよく扱う．この `[a]` は `a` という変数 1 要素からなるリストのこともあるし，`a` 型という仮の型から作ったリスト型の場合もあるが，字面からでは判断できない．もし変数はイタリック体，型はボールド体と決まっていれば，それぞれ $[a]$ および $[\boldsymbol{a}]$ と区別できたところである．

本書は，異なる性質のものには異なる書体を割り当てるようにしている．ただし，どの表現もいつでも Haskell に翻訳できるように配慮している．実際，本書執筆の最大の困難点は，数学的に妥当で，かつ Haskell の記法とも矛盾しない記法を見つけることであった．

3 関数型という考え方

TK

```
$ cat the-great-gatsby.txt | tr '[A-Z]' '[a-z]' | tr -C -d '[a-z ]' | tr ' ' '\n'
| sort | uniq -c | sort -nr
```

$$
\begin{aligned}
f_1 &= \mathrm{tr}_{[A\ldots Z]\to[a\ldots z]} \\
f_2 &= \mathrm{tr}_{\overline{[a\ldots z,\square]}\to\emptyset} \\
f_3 &= \mathrm{tr}_{\square\to\swarrow} \\
f_4 &= \mathrm{sort} \\
f_5 &= \mathrm{uniq}_c \\
f_6 &= \mathrm{sort}_{n,r}
\end{aligned} \tag{1}
$$

$$y = f_6(f_5(f_4(f_3(f_2(f_1(x)))))) \tag{2}$$

$$y = f_6 \bullet f_5 \bullet f_4 \bullet f_3 \bullet f_2 \bullet f_1(x) \tag{3}$$

4 変数・関数・型

4.1 変数

変数 x に値 1 を代入するには次のようにする．

$$x = 1 \tag{4}$$

- Haskell では

```haskell
x = 1
```

と書く．

変数という呼び名に反して，変数の値は一度代入したら変えられない．そこで変数に値を代入するとは呼ばずに，変数に値を束縛するという． 式 4 の右辺のように数式にハードコードされた値をリテラルと呼ぶ．

リテラルや変数には型がある．型は数学者の集合と似た意味で，整数全体の集合 $\mathbb{Z}$ に相当する整数型や，実数全体の集合 $\mathbb{R}$ に相当する浮動小数点型がある．整数と整数型，実数と浮動小数点型は異なるため，整数型を **Int** で，浮動小数点型を **Double** で表すことにする．

- Haskell では **Int** および **Double** をそれぞれ `Int` および `Double` と書く．

数学者は変数 x が整数であることを $x \in \mathbb{Z}$ と書くが，本書では $x :: \mathbf{Int}$ と書く．これは記号 $\in$ を別の用途に用いるためである．

- Haskell では $x :: \mathbf{Int}$ を

```haskell
x :: Int
```

と書く．

本書では変数名を原則 1 文字として，イタリック体で表し w, x, y, z のような n 以降のアルファベットを使う．

変数の値がいつでも変化しないことを参照透過性と呼ぶ．プログラマが変数の値を変化させたい，つまり破壊的代入を行いたい理由はユーザ入力，ループ，例外，内部状態の変化，大域ジャンプ，継続を扱いたいからであろう．しかし，後に見るようにループ，例外，内部状態の変化，大域ジャンプ，継続に変数の破壊的代入は必要ない．ユーザ入力に関しても章を改めて取り上げる．参照透過性を強くサポートするプログラミング言語をを関数型プログラミング言語と呼ぶ．

4.2 変数の型

TK

$$x :: \mathbf{Int} \tag{5}$$

$$x = 1 \tag{6}$$

$$x :: \mathbf{Int} = 1 \tag{7}$$

4.3 関数

整数 x に 1 を足す関数 f は次のように定義できる．

$$fx = x + 1 \tag{8}$$

ここに x は関数 f の引数である．引数は括弧でくるまない．

- Haskell では $fx = x + 1$ を

```haskell
f x = x + 1
```

と書く．

本書では関数名を原則 1 文字として，イタリック体で表し，f, g, h のようにアルファベットの f 以降の文字を使う．ただし有名な関数についてはローマン体で表し，文字数も 2 文字以上とする．たとえば sin などの三角関数や指数関数がそれにあたる．

変数 x に関数 f を適用する場合は次のように書く．ここでも引数を括弧でくるまない．

$$z = fx \tag{9}$$

- Haskell では $z = fx$ を

```
z = f x
```

と書く．

関数 f が引数をふたつ取る場合は，次のように書く．

$$z = fxy \tag{10}$$

- Haskell では $z = fxy$ を

```
z = f x y
```

と書く．

なお fxy は $(fx)y$ と解釈される．前半の (fx) は 1 引数の関数とみなせる．2 引数関数を連続した 1 引数関数の適用とみなす考え方を，関数のカリー化と呼ぶ．

4.4 有名な関数

TK

4.5 関数の型

TK

4.6 関数合成

変数 x に関数 f と関数 g を連続して適用したい場合 $z = g(fx)$ とするところであるが，事前に関数 f と関数 g を合成しておきたいことがある．

関数の合成は次のように書く．

$$k = g \bullet f \tag{11}$$

関数の連続適用 $g(fx)$ と合成関数の適用 $(g \bullet f)x$ は同じ結果を返す．

- Haskell では $k = g \bullet f$ を

```
k = g . f
```

と書く．

関数合成演算子 . は以下のように左結合する．

$$\begin{aligned} k &= h \bullet g \bullet f \\ &= (h \bullet g) \bullet f \end{aligned} \tag{12}$$

関数適用のための特別な演算子 § があると便利である．演算子 § は関数合成演算子よりも優先順位が低い．例を挙げる．

$$\begin{aligned} z &= h \bullet (g \bullet f)x \\ &= h \,\S\, g \bullet fx \end{aligned} \tag{13}$$

- Haskell では $z = h \,\S\, g \bullet fx$ を

```
z = h $ (g . f) x
```

と書く．

4.7 IO サバイバルキット 1

プログラムとは合成された関数である．多くのプログラミング言語では，プログラムそのものに main という名前をつける．本書では「IO モナド」の章で述べる理由によって，main 関数をサンセリフ体で main と書く．

実用的なプログラムはユーザからの入力を受け取り，関数を適用し，ユーザへ出力する．Haskell ではユーザからの 1 行の入力を getLine で受け取り，変数の値を print で書き出せる．ここに getLine と print は関数（ファンクション）ではあるが，特別に「アクション」とも呼ぶ．関数 main もアクションである．

引数 x の 2 乗を求める関数 f は次のように定義できる．

$$\begin{aligned} f &:: \mathbf{Double} \to \mathbf{Double} \\ fx &= x \times x \end{aligned} \tag{14}$$

ユーザからの入力に関数 f を適用してユーザへ出力するプログラムを Haskell で書くと次のようになる．

$$\mathsf{main} = \mathsf{print} \bullet f \bullet \mathrm{read} \heartsuit \mathsf{getLine} \tag{15}$$

ここに関数 read は文字列であるユーザ入力を数に変換する関数である．また演算子 ♡ は新たな関数合成演算子で，アクションとアクションを合成するための特別な演算子である．詳細は「モナド」の章で述べる．

- Haskell では 式 14 と式 15 をまとめて

```
f :: Double -> Double
f x = x * x

main = print . f . read =<< getLine
```

と書く．

5 条件分岐と再帰呼び出し

5.1 ラムダ

関数とは，変数名に束縛されたラムダ式である．引数をひとつとり，その引数に 1 を足して返す関数 f はラムダ式を用いて次のように書ける．

$$f = \backslash\, x \mapsto x + 1 \tag{16}$$

ラムダ記号は一般的には λ が用いられるが，本書ではすべてのギリシア文字を予約しておきたいので，Haskell の記法に倣って \ を用いる．

- Haskell では $f = \backslash\, x \mapsto x + 1$ を

```haskell
f = \x -> x + 1
```

と書く．

ラムダ式は入れ子に出来る．引数をふたつとり，その引数同士を足すラムダ式は次のように書ける．

$$\backslash\, x \mapsto (\backslash\, y \mapsto x + y) \tag{17}$$

ラムダ式中の $\mapsto$ は右結合するので，式 17 は次のようにも書ける．

$$\backslash\, x \mapsto \backslash\, y \mapsto x + y \tag{18}$$

より簡潔に式 18 を

$$\backslash\, xy \mapsto x + y \tag{19}$$

と書いても良い．

本書では無名変数 $\lozenge$ を用いた以下の書き方も用いる．

$$\begin{aligned} f &= (\lozenge + 1) \\ &= \backslash\, x \mapsto x + 1 \end{aligned} \tag{20}$$

- 無名変数は Haskell には無いが，代わりに「セクション」という書き方ができる．式 $f = (\lozenge + 1)$ を Haskell では

```haskell
f = (+1)
```

と書く．

無名変数が 2 回以上登場した場合は，その都度新しいパラメタを生成する．たとえば次のとおりである．

$$\begin{aligned} f &= \lozenge + \lozenge \\ &= \backslash\, x \mapsto \backslash\, y \mapsto x + y \\ &= \backslash\, xy \mapsto x + y \end{aligned} \tag{21}$$

- Haskell では $f = (\lozenge + \lozenge)$ を

```haskell
f = (+)
```

と書く．

5.2 ローカル変数

関数内でローカル変数を使いたい場合は以下のように行う．

$$z = \texttt{let}\ y \equiv 1\ \texttt{in}\ x + y \tag{22}$$

- Haskell では $z = \texttt{let}\ y \equiv 1\ \texttt{in}\ x + y$ を

```haskell
z = let y = 1 in x + y
```

と書く．

ローカル変数はラムダ式のシンタックスシュガーである．式 22 は次の式と等価である．

$$z = (\backslash\, y \mapsto x + y)1 \tag{23}$$

ローカル変数の定義は次のように後置できる．

$$z = x + y \texttt{ where } y \equiv 1 \tag{24}$$

- Haskell では $z = x + y \texttt{ where } y \equiv 1$ を

```
z = x + y where y = 1
```

と書く．

5.3 クロージャ

ラムダ式を返す関数は，ラムダ式内部に値を閉じ込めることができる．たとえば

$$fn = \backslash\, x \mapsto n + x \tag{25}$$

のように関数を定義して良い．関数 f に引数 n を与えると，新たな 1 引数関数が得られる．例を挙げる．

$$\begin{aligned} n &= 3 \\ g &= fn \end{aligned} \tag{26}$$

この例では，関数 g の中に値 $n = 3$ が閉じ込められているため $g1$ は 4 と評価される．値を閉じ込めたラムダ式をクロージャと呼ぶ．

5.4 型

TK 一部前倒し

すべての変数，関数には型がある．代表的な型には整数型，浮動小数点型，ブール型，文字型がある．整数型を **Int** で，浮動小数点型を **Double** で表すことはすでに述べたとおりである．

Haskell には 2 種類の整数型がある．ひとつは固定長整数型で，もうひとつは多倍長整数型である．Haskell では前者を `Int` で，後者を `Integer` で表す．多倍長整数型はメモリの許す限り巨大な整数を扱えるので，整数全体の集合に近いのであるが，本書では主に固定長整数型を用いる．

浮動小数点型には単精度浮動小数点型と倍精度浮動小数点型があり，Haskell では前者を `Float` で，後者を `Double` で表現する．単精度浮動小数点型はめったに用いられないため，本書では浮動小数点型と言えば倍精度浮動小数点型を意味することにする．

論理型は論理値 True または False のいずれかしか値をとれない型である．論理型のことを **Bool**と書く．

- Haskell ではブール型を `Bool` と書く．

6 Test Part

Function application.

$$z = fx \tag{27}$$

```
z = f x
```

List map.

$$z_{\mathrm{s}} = f \star x_{\mathrm{s}} \tag{28}$$

```
zs = f `map` xs
```

Functor map.

$$z_* = f * x_* \tag{29}$$

```
zm = f <$> xm
```

Applicative map.

$$z_* = f_* \circledast x_* \tag{30}$$

```
zm = fm <*> xm
```

Apllicative map 2.

$$z_* = g_* \circledast x_* \circledast y_* \tag{31}$$

```
zm = gm <*> xm <*> ym
```

Applicative map 2 (another version).

$$z_* = g * x_* \circledast y_* \tag{32}$$

```
zm = g <$> xm <*> ym
```

Monadic function application.

$$z_* = f^\dagger x \tag{33}$$

```
zm = f x
```

Bind.

$$z_* = f^\dagger \heartsuit x_* \tag{34}$$

```
zm = f =<< xm
```

Double bind.

$$z_* = g^\dagger \heartsuit f^\dagger \heartsuit x_* \tag{35}$$

```
zm = g =<< f =<< xm
```